Machine Translated by Google

Euromed University of Fez — UEMF

Multi-Agent Systems — JADE

Lab Report 02

ACL communication between Agents

Prof. Abderrahim Waga

Academic year 2025–2026

Machine Translated by Google

Table of Contents

1 Introduction 2

2 Exercise 1 — Unidirectional Communication 2.1 Objective . 2
. 2
2.2 Implementation 2
2.3 Console output 2
2.4 Key points 3

3 Exercise 2 — Two-Way Communication 3.1 Objective . 4
. 4
3.2 Implementation 4
3.3 Console output 4
3.4 Key points 5

Exercise 3 — Coordination of 3 Agents (1 to 2) 6
4.1 Objective 6
4.2 Implementation 6
4.3 Console output 7
4.4 Key points 7

5 Exercise 4 — Decision Making and Negotiation 5.1 Objective . 8
. 8
5.2 Implementation 8
5.3 Console output 9
5.4 Key points 10

6 Conclusion 10

Machine Translated by Google

Introduction

This report presents exercises 1, 2 and 3 of the practical work on ACL (Agent Com-munication Language) communication in the JADE (Java Agent DEvelopment Framework) framework.

The goal is to understand how agents exchange structured messages at using standard performatives (INFORM, REQUEST) and mechanisms like createReply() to coordinate their interactions.

Exercise 1 — One-Way Communication

Objective

Agent A sends an INFORM message to Agent B. Agent B receives and displays the message. without sending a reply.

Implementation

AgentA uses a OneShotBehaviour to send a single message at startup:

Listing 1 – AgentA — INFORM submission

```
addBehaviour (new OneShotBehaviour() {
        public void action() {
                ACLMessage msg = new ACLMessage ( ACLMessage . INFORM );
                msg. addReceiver (new AID ("agent1 ", AID.ISLOCALNAME));
                msg. setContent ("Hello B!");
                send(msg);
                System. out. println(getLocalName() + " sent message to agentB ");

        }
}) ;
```

AgentB uses a CyclicBehaviour to continuously listen for incoming messages:

Listing 2 – AgentB — reception and display

```
addBehaviour (new CyclicBehaviour() {
        public void action() {
                ACLMessage msg = receive();
                if (msg != null ) {
                        System. out. println ("From: " + msg.getSender().
                            getLocalName());
                        System. out. println ("Content: " + msg.getContent();
                }
        }
}) ;
```

Console result

Machine Translated by Google


```
agentA started and will send a message ... agentA sends message to agentB agent1
received message: From: agentA Content: Hello B!
```

Key points

— OneShotBehaviour: executed only once, suitable for a one-off sending.
— CyclicBehaviour: infinite loop listening for incoming messages.
— AID: unique identifier of the recipient agent in the JADE platform.
— Good practice: call block() when msg == null to avoid a busy-wait.

Machine Translated by Google

Exercise 2 — Two-Way Communication

Objective

Agent A sends a request (REQUEST) to Agent B. Agent B processes the request and returns a AgentA receives and displays the response (INFORM) containing the temperature.

Implementation

AgentA modifies the performative to REQUEST and adds a CyclicBehaviour to listen The answer:

Listing 3 – AgentA — sending REQUEST + listening for response

```
// Sending the request
ACLMessage msg = new ACLMessage ( ACLMessage . REQUEST );
messages. addReceiver (new AID ("agent1 ", AID.ISLOCALNAME));
messages. setContent ("What is the temperature ?");
send ( msg ) ;

// Listening to the response
addBehaviour (new CyclicBehaviour() {
        public void action() {
                ACLMessage msg = receive();
                if (msg != null ) {
                        System. out. println ("From: " + msg.getSender().
                            getLocalName());
                        System. out. println ("Content: " + msg.getContent();
                } else { block
                (); }
        }
}) ;
```

AgentB checks the performative and uses createReply() to generate the response:

Listing 4 – AgentB — REQUEST processing and INFORM response

```
if ( msg . getPerformative () == ACLMessage . REQUEST ) {
        ACLMessage reply = msg . createReply();
        reply . setPerformative(ACLMessage.INFORM);
        reply . setContent ("It is 25 degrees C.");
        send ( reply );
        System. out. println(getLocalName() + " sent reply to " + msg .
                getSender(). getLocalName () );
}
```

Console result

```
Agent A sent a message to Agent B.
agentA waiting for response ...
agent1 received: REQUEST -- What is the temperature?
agent1 sent reply to agentA
agentA received: INFORM -- It is 25 degrees C.
```

Machine Translated by Google

Key points

— createReply() automatically copies the metadata (conversation-id, reply-to) to ensure the consistency of the exchange.
— Checking getPerformative() allows the agent to filter and process only the relevant messages.
— AgentA separates responsibilities with two independent behaviors: one for one for sending, one for receiving.

Machine Translated by Google

Exercise 3 — Coordination of 3 Agents (1 to 2)

Objective

A CoordinatorAgent sends REQUEST requests to two expert agents (Expert1 and Expert2), waits for their two INFORM responses, then displays an overall result and deletes his listening behavior.

Implementation

CoordinatorAgent sends two requests and then listens for the responses with a counter:

Listing 5 – Coordinator Agent — dispatch and coordination

```
private int counter = 0;

// Sending requests
addBehaviour (new OneShotBehaviour() {
        public void action() {
                ACLMessage msg = new ACLMessage ( ACLMessage . REQUEST );
                msg. addReceiver (new AID ("agent1 ", AID.ISLOCALNAME));
                msg. setContent ("Hello B!");
                send(msg);

                ACLMessage msg2 = new ACLMessage (ACLMessage. REQUEST);
                msg2 . addReceiver( new AID ("agent2 ", AID.ISLOCALNAME));
                msg2 . setContent ("Hello C!") ;
                send(msg2);
        }
}) ;

// Listening to responses with a counter
addBehaviour (new CyclicBehaviour( this ){
        public void action() {
                ACLMessage msg = receive();
                if (msg != null ) {
                        if ( msg . getPerformative () == ACLMessage . INFORM ) {
                                counter ++;
                                System. out. println ("Received INFORM from +                                "
                                        msg.getSender().getLocalName());
                                if ( counter == 2) {
                                        System. out. println ("All replies received!") ;
                                        myAgent . removeBehaviour( this );
                                }
                        }
                } else { block () ; }
        }
});
```

Expert1 and Expert2 function identically to AgentB in exercise 2: they
They receive a REQUEST and respond with an INFORM via createReply():

Machine Translated by Google

Listing 6 – Expert1 / Expert2 — INFORM response

```
if ( msg . getPerformative () == ACLMessage . REQUEST ) {
        ACLMessage reply = msg . createReply();
        reply . setPerformative(ACLMessage.INFORM);
        reply . setContent ("Hello Coordinator! I am "
                + getLocalName () + send " I received your request .
        ( reply );
}
```

Console result

```
coordinator started and will send messages ...
coordinator sends messages to Expert1 and Expert2
agent1 received: REQUEST -- Hello B!
agent2 received: REQUEST -- Hello C!
agent1 sends reply to coordinator
agent2 sends reply to coordinator
coordinator received INFORM from agent1 | Counter = 1
coordinator received INFORM from agent2 | Counter = 2
========================================

OVERALL RESULT: All replies completed!
Total INFORM received: 2
========================================

coordinator: Listener behavior removed.
```

Key points

— An instance counter allows you to track the number of responses received.

— myAgent.removeBehaviour(this) cleanly stops the listening behavior once the coordination is complete.

— Expert1 and Expert2 are symmetrical: same logic, different names. We would have I could use a single parameterized class.

— The OneShotBehaviour and the CyclicBehaviour run in parallel in JADE thanks to the agent's internal scheduler.

Machine Translated by Google

Exercise 4 — Decision Making and Negotiation

Objective

A Buyer agent issues a Request for Proposals (RFP) to Seller1 and Seller2. The sellers submit their prices (PROPOSE). The Buyer compares the prices and sends an ACCEPT_PROPOSAL to the cheaper and a REJECT_PROPOSAL to the other.

Implementation

Buyer — Phase 1: Sending the CFPs to both sellers:

Listing 7 – Buyer — CFP shipment

```
ACLMessage cfp1 = new ACLMessage ( ACLMessage . CFP );
cfp1. addReceiver (new AID ("seller1 ", AID.ISLOCALNAME));
cfp1. setContent ("Call for proposal - Please send your price ");
send ( cfp1 );

ACLMessage cfp2 = new ACLMessage ( ACLMessage . CFP );
cfp2. addReceiver (new AID ("seller2 ", AID.ISLOCALNAME));
cfp2. setContent ("Call for proposal - Please send your price ");
send ( cfp2 );
```

Buyer — Phase 2: Receiving proposals and making a decision. A MessageTemplate filters only the PROPOSED messages. After receiving both proposals, the Buyer compares them.
and replies:

Listing 8 – Buyer — comparison and decision

```
MessageTemplate proposesTemplate =
        MessageTemplate.MatchPerformative(ACLMessage.PROPOSE);

ACLMessage msg = receive ( proposeTemplate );
if (msg != null ) {
        int price = Integer. parseInt(msg.getContent());
        proposalCount ++;
        if ( price < bestPrice ) {
                bestPrice = price;
                bestSeller = msg . getSender();
        }
        if (proposalCount == 2) {
                // Send ACCEPT to the winner
                ACLMessage accept = winnerProposal . createReply();
                accept. setPerformative ( ACLMessage . ACCEPT_PROPOSAL );
                accept. setContent ("Congratulations! You won the deal.") ;
                send ( accept );

                Send REJECT to the loser
                ACLMessage reject = loserProposal . createReply();
                reject . setPerformative ( ACLMessage . REJECT_PROPOSAL );
                reject . setContent ("Sorry, your price was too high .");
                send ( reject ) ;
```

Machine Translated by Google


```
24
25                  removeBehaviour( this ); // stop listening
26          }
27 }
```

Seller 1 / Seller 2 — the salespeople are symmetrical. They listen to the CFPs, respond with PROPOSE, then await the final decision:

Listing 9 – Seller — CFP response and pending decision

```
1 // Respond to the CFP
2 ACLMessage proposes = msg . createReply();
3 proposes. setPerformative ( ACLMessage . PROPOSED );
4 suggests. setContent(String.valueOf(myPrice));
5 send ( propose );
6
7 // Wait for ACCEPT or REJECT
8 MessageTemplate decisionTemplate = MessageTemplate . gold (
9           MessageTemplate. MatchPerformative ( ACLMessage . ACCEPT_PROPOSAL )                    ,
10          MessageTemplate. MatchPerformative (ACLMessage.REJECT_PROPOSAL)
11 );
12 ACLMessage decision = blockingReceive ( decisionTemplate );
13
14 if ( decision . getPerformative () == ACLMessage . ACCEPT_PROPOSAL ) {
15          System. out. println (sellerName + ": Hooray, I won the deal!") ;
16 } else {
17          System. out. println (sellerName + ": Sorry,                    I lost the deal .
18 }
```

Console result

```
buyer (Buyer) started ...
seller1 started with price: 500
seller2 started with price: 450
buyer sent CFP to Seller1
buyer sent CFP to Seller2
seller1 received CFP from buyer => sent PROPOSED: 500
seller2 received CFP from buyer => sent PROPOSED: 450
buyer received PROPOSE from seller1 with price: 500
buyer received PROPOSE from seller2 with price: 450
========================================

Comparison:
    Seller1 price: 500
    Seller2 price: 450
    Winner: seller2 with price 450
========================================

buyer sends ACCEPT_PROPOSAL to seller2
buyer sends REJECT_PROPOSAL to seller1
seller2: Hooray, I won the deal! Price = 450
seller1: Sorry, buyer:          I lost the deal. Better luck next time!
Decision made                   ,    listener stopped.
```

Machine Translated by Google

Key points

— MessageTemplate.MatchPerformative() filters messages by type, avoiding processing irrelevant messages.
— Integer.MAX_VALUE as the initial price guarantees that the first bid received becomes automatically the best reference price.
— blockingReceive() in Sellers suspends the agent until a message is received sage matching the template, without busy-wait.
— Seller1 and Seller2 are structurally identical; in practice, a single class A configured setting would suffice.
— The price can be passed as a JADE argument at startup, making the agents confident-manageable without recompilation.

Conclusion

These three exercises progressively cover the fundamental ACL communication mechanisms in JADE:

— Exercise 1: simple unidirectional sending with INFORM.
— Exercise 2: bidirectional request/response exchange with REQUEST and createReply().
— Exercise 3: Multi-agent coordination with state management (counter) and dynamic suppression of behavior.
— Exercise 4: Negotiation with CFP/PROPOSE, price comparison and decision (ACCEPT_PROPOSAL / REJECT_PROPOSAL).

These four exercises form a coherent progression: from simple message sending to multi-agent negotiation. The ACL mechanisms covered provide the foundation for designing robust multi-agent systems.